

MDL-004 — Interaction Model

External publication draft

MDL-004 — Interaction Model

Understanding explains the world. Interaction explains how the world changes.

Purpose

MDL-001 established **why** Mosaic exists.

MDL-002 established **how** design decisions are made.

MDL-003 established **how Mosaic understands the world**.

MDL-004 defines **how that world behaves over time**.

Where the Mental Model describes concepts, the Interaction Model describes behaviour.

It explains:

- how Focus changes
- how Context evolves
- how Composition adapts
- how movement communicates understanding
- how continuity is preserved
- how users travel through their World without leaving it

Relationship to Previous Specifications

Vision

↓

Principles

↓

Mental Model

↓

Interaction Model

↓

Composition Model

↓

Design System

↓

Implementation

The Interaction Model assumes the concepts introduced by the Mental Model already exist.

Its responsibility is to define how those concepts evolve during interaction.

Scope

This specification defines:

- Behaviour
- Continuity
- Focus transitions
- Context transitions
- Temporal interaction
- Adaptive composition
- User flow
- Interaction hierarchy

This specification intentionally does **not** define:

- Motion timing
- Animation curves
- Materials
- Components
- Typography
- Layout
- Rendering

Those concerns belong to MDS.

Core Question

MDL-004 exists to answer one question.

How should Mosaic behave?

Not:

How should it look?

Behaviour Statement

Users should never feel they have navigated between disconnected pages.

Instead they should feel that:

Their World has naturally reorganised itself around what now matters.

Everything within this specification reinforces that single idea.

Primary Behaviour Model

The Interaction Model introduces the following behavioural pipeline.

World



Focus Change



Context Update



Composition Re-evaluation



Expression Selection



Presentation Update

Interaction is therefore defined as:

The continuous evolution of the user's World.

Expected Outcome

After reading MDL-004, contributors should understand:

- how Mosaic changes over time
- why movement exists
- why compositions evolve
- how continuity is preserved
- how interaction differs from navigation
- why adaptive behaviour exists

without discussing implementation.

Repository Structure

design/

■■■ mdl/

■■■ MDL-004 Interaction Model/

README.md

00-document-control.md

01-what-is-an-interaction-model.md

02-continuity.md

03-focus-transitions.md

04-context-transitions.md

05-composition-evolution.md

06-movement.md

07-user-flow.md

08-temporal-behaviour.md

09-interaction-states.md

10-user-vs-system-behaviour.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model

Downstream specifications:

- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-005 Motion System
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDL-004
Title	Mosaic Design Language — Interaction Model
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 Vision, MDL-002 Principles, MDL-003 Mental Model
Repository	/design/mdl/MDL-004 Interaction Model/

Purpose

MDL-004 defines the behavioural architecture of Mosaic.

Where the Mental Model establishes **what exists**, the Interaction Model establishes **how those concepts evolve over time**.

It defines the expected behaviour of the platform independently from:

- visual appearance
- rendering technology
- frontend framework
- animation implementation
- platform-specific interaction

This separation ensures that interaction remains conceptually consistent regardless of implementation.

Authority

MDL-004 governs behavioural architecture.

Its authority extends to:

- User Experience
- Interaction Design
- Motion Philosophy
- Runtime Behaviour
- Composition Behaviour

- Navigation Behaviour
- Extension Behaviour

MDL-004 intentionally does **not** define:

- animation timing
- easing curves
- visual styling
- typography
- materials
- spacing

Those concerns belong to the Mosaic Design System.

Relationship To MDL

The Interaction Model occupies a unique position within the Design Language.

[mermaid]
flowchart TD

Vision["MDL-001 Vision"]

Principles["MDL-002 Principles"]

Mental["MDL-003 Mental Model"]

Interaction["MDL-004 Interaction Model"]

Composition["MDL-005 Composition Model"]

DesignSystem["MDS Specifications"]

Implementation["Product"]

Vision --> Principles
Principles --> Mental
Mental --> Interaction
Interaction --> Composition
Composition --> DesignSystem
DesignSystem --> Implementation

The Interaction Model translates conceptual understanding into observable behaviour.

It answers:

What happens when the user's World changes?

Design Intent

Traditional applications are built around navigation.

Users move:

Page

↓

Page



Page

Mosaic intentionally rejects this model.

Instead:

The World remains.

Focus changes.

Context evolves.

Composition reorganises.

Presentation updates.

The user experiences continuity rather than relocation.

Behavioural Scope

This specification governs the behaviour of:

- Focus
- Context
- Composition
- Expressions
- Navigation
- Discovery
- Interaction States
- Runtime Adaptation

It intentionally avoids implementation-specific discussion.

Reader Expectations

Before continuing, contributors should already understand:

- Why Mosaic exists.
- How design decisions are made.
- How Mosaic understands the user's World.

MDL-004 assumes familiarity with:

- MDL-001
- MDL-002
- MDL-003

This document does not redefine those concepts.

Instead, it explains how they behave.

Stability

Interaction behaviour should evolve more slowly than implementation.

Expected lifetime:

Artefact	Expected Lifetime
Components	Months
Motion Curves	Months
Behaviour	Years
Mental Model	Decades

This distinction allows Mosaic to modernise visually while preserving behavioural consistency.

Success Criteria

MDL-004 succeeds when:

- users rarely become disoriented
- movement always feels intentional
- interaction reinforces understanding
- navigation feels increasingly unnecessary
- contributors naturally describe behaviour using MDL terminology

Interaction should become understandable before it becomes beautiful.

Review Status

Status

Draft

Dependencies

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model

Supersedes

None.

Next File

01-what-is-an-interaction-model.md

What Is An Interaction Model?

Purpose

Before defining how Mosaic behaves, contributors must first understand what an Interaction Model is.

Many software projects unintentionally treat interaction as a collection of animations, transitions and user inputs.

Within the Mosaic Design Language, interaction is something much broader.

Interaction defines **how the user's World evolves over time**.

It is the behavioural layer that connects the conceptual architecture defined by the Mental Model with the visual implementation defined by the Design System.

Definition

Within MDL, an **Interaction Model** is defined as:

The behavioural rules that govern how the user's World changes in response to time, intent and interaction.

Interaction is not:

- animation
- navigation
- gestures
- buttons
- transitions

Those are implementations of interaction.

Interaction itself describes behavioural change.

Why An Interaction Model Exists

A user continually asks the platform questions.

Often without consciously realising it.

Questions such as:

- Where am I?
- What changed?
- What should I look at?
- What happens next?
- Why did that move?

- Can I get back?

A good Interaction Model answers those questions naturally.

Without explanation.

Without tutorials.

Without requiring users to consciously think about software.

Interaction Is Behaviour

Interaction should be understood as behaviour rather than interface.

Example.

Poor mental model.

Click Button

↓

Animation

↓

New Screen

Mosaic mental model.

Intent

↓

Focus Changes

↓

Context Evolves

↓

Composition Updates

↓

Presentation Reflects

Notice that interface appears only at the very end.

Behaviour exists independently from interface.

Interaction Begins Before Input

Interaction does not begin when the user clicks.

Interaction begins the moment the user's World changes.

Examples include:

- playback starts
- playback ends

- next episode releases
- reading progress updates
- music changes
- a plugin contributes new information

These events may occur:

- because of user action
- because of time
- because of external information
- because of another device

The Interaction Model governs them all consistently.

Behaviour Before Motion

Movement communicates behaviour.

Movement is not behaviour itself.

For example.

Behaviour:

Focus changes

Possible implementations:

- slide
- dissolve
- expand
- reduce
- no animation

The behaviour remains identical.

Only presentation changes.

This distinction allows Mosaic to evolve visually without changing how users understand the platform.

Interaction Is Continuous

Traditional applications often behave like this.

Page

↓

Page

↓

Page

↓

Page

Every action discards the previous state.

Mosaic intentionally behaves differently.

World

↓

Focus

↓

Context

↓

Composition

↓

Updated World

Nothing is discarded.

The World simply evolves.

Interaction Is Intentional

Every behavioural change should have a reason.

Users should never wonder:

- Why did that appear?
- Why did that disappear?
- Why did everything move?
- Why am I somewhere else?

Instead they should instinctively understand:

"The platform changed because my World changed."

Understanding creates trust.

Trust reduces cognitive effort.

Interaction Preserves Continuity

The primary objective of the Interaction Model is preserving continuity.

Users should feel:

- progression
- evolution
- continuity

Not:

- interruption
- relocation
- fragmentation

The interface should continuously reassure users that they remain inside the same World.

Only its current state has evolved.

Interaction Has Memory

Every interaction begins with a previous state.

Previous World

↓

User Intent

↓

Updated World

The previous state should always influence the next.

Interaction should therefore appear cumulative rather than isolated.

This continuity allows users to build long-term confidence in how Mosaic behaves.

Good Examples

Example 01

User finishes an episode.

Instead of:

Episode Ends

↓

Homepage

Mosaic behaves:

Episode Ends

↓

Progress Updates

↓

Next Episode Gains Priority

↓

Composition Evolves

The user never leaves their current World.

Example 02

User changes from Anime to Books.

The interface should communicate:

Your attention has changed.

Not:

You launched another application.

The World remains.

Only Focus changes.

Anti-patterns

The following behaviours violate the Interaction Model.

Teleportation

Content instantly appears elsewhere.

Continuity is lost.

Page Thinking

Every interaction replaces one complete interface with another.

The World disappears.

Animation Without Meaning

Movement exists purely for visual effect.

No additional understanding is created.

Context Reset

Changing one small piece of information causes the entire experience to restart.

The user loses orientation.

Interaction Hierarchy

Interaction always follows the same conceptual sequence.

[mermaid]
flowchart TD

Intent
Intent --> Focus

Focus --> Context
Context --> Composition
Composition --> Expression
Expression --> Presentation

Interaction never bypasses intermediate layers.

This consistency allows users to gradually build an accurate understanding of the platform.

Relationship To Future Specifications

This chapter establishes the behavioural philosophy.

Future chapters define:

- continuity
- focus transitions
- context transitions
- temporal behaviour
- interaction states
- composition evolution

Together these chapters form the complete behavioural architecture of Mosaic.

Summary

The Interaction Model defines how the user's World changes.

It is not concerned with:

- visuals
- animation
- components

Instead it defines:

- behaviour
- continuity
- evolution
- understanding

The interface communicates those behaviours.

It does not invent them.

Review Status

Status

Draft

Next File

02-continuity.md

Continuity

Purpose

Continuity is the primary behavioural objective of the Mosaic Interaction Model.

Users should never feel that they are repeatedly leaving one interface and entering another.

Instead, they should experience one continuous entertainment world that naturally evolves around their current attention.

Every interaction defined by MDL should reinforce continuity.

Definition

Within MDL, **Continuity** is defined as:

The preservation of the user's understanding as their World evolves.

Continuity answers one question.

"Do I still understand where I am?"

If the answer is yes...

The interaction has succeeded.

Why Continuity Exists

Traditional applications organise themselves around destinations.

Home

↓

Series

↓

Season

↓

Episode

↓

Player

Each destination discards the previous one.

Users repeatedly reconstruct their mental model.

Mosaic intentionally avoids this behaviour.

Instead:

World

↓

World evolves

↓

World evolves

↓

World evolves

The user remains inside the same conceptual environment.

Only emphasis changes.

The Illusion Of One World

The user should rarely feel that they have "gone somewhere."

Instead they should feel that:

- something became more important
- something became less important
- something new entered the World
- something completed
- something naturally moved aside

These changes should feel inevitable.

Not surprising.

Continuity Before Navigation

Navigation is an implementation.

Continuity is a behavioural objective.

When contributors ask:

"Where should this page live?"

they should instead ask:

"How does the user's World naturally evolve to reveal this?"

This subtle distinction changes almost every interaction.

Continuity Preserves Memory

People rapidly build spatial and conceptual memory.

Examples include:

- where information usually appears

- what currently matters
- how they reached their current state

Breaking continuity forces users to rebuild that understanding.

Every unnecessary rebuild increases cognitive effort.

The purpose of continuity is therefore preserving accumulated understanding.

Small Changes

Minor changes should produce minor behavioural evolution.

Examples include:

Episode Progress

↓

Progress Updates

↓

Next Episode Gains Importance

The World barely changes.

Only local understanding evolves.

Medium Changes

Changing Focus within the same Domain.

Example.

Frieren

↓

Fire Force

The Domain remains:

Anime

The World therefore changes moderately.

Users should recognise:

- familiar structure
- familiar relationships
- familiar interaction

Only the Focus has changed.

Large Changes

Changing Domains.

Example.

Anime

↓

Books

Now:

- Relationships change
- Information changes
- Expressions change
- Composition changes

The behavioural transition should therefore feel more substantial.

Importantly...

The World still remains continuous.

The user has not "launched another application."

Behavioural Weight

Every interaction possesses behavioural weight.

Weight is determined by conceptual distance.

Not implementation.

Examples.

Transition	Behavioural Weight
Progress Update	Minimal
Episode → Episode	Low
Series → Series	Medium
Anime → Books	High
Entertainment → Administration	Very High

Future specifications use behavioural weight when defining interaction and composition behaviour.

Progressive Evolution

Continuity should favour gradual evolution.

Poor.

Everything changes.

↓

User understands nothing.

Better.

One thing changes.

↓

User understands.

↓

Next thing changes.

↓

User understands.

↓

World evolves.

The objective is preserving orientation throughout the interaction.

State Preservation

Continuity requires preserving meaningful state.

Examples include:

- scroll position
- reading progress
- playback progress
- expanded information
- selected Focus

Users should rarely need to reconstruct work they have already completed.

Good Examples

Example 01

Playback Ends.

Instead of:

Player closes.

↓

Homepage.

Mosaic behaves:

Playback Ends.

↓

Progress Updates.



Next Episode Becomes Focus.



Composition Evolves.

The user never loses their place.

Example 02

Reading Ends.

Instead of:

Book closes.



Library.

Mosaic behaves:

Book Complete.



Series Progress Updates.



Next Book Gains Importance.



Relationships Expand.

Again...

The World evolves.

Anti-patterns

Hard Navigation

Replacing one complete experience with another.

Context Loss

The user returns to an unrelated interface after completing an activity.

Behaviour Reset

The interface forgets:

- Focus

- Context
- Progress

without reason.

Arbitrary Movement

Elements move without conceptual justification.

Understanding decreases.

Continuity Across Devices

Continuity should extend beyond a single device.

Example.

User begins:

Reading

↓

Tablet

Later continues:

Phone

The World should feel identical.

Presentation changes.

Continuity remains.

Future implementation should preserve conceptual continuity even when interaction models differ between platforms.

Plugins

Extensions should preserve continuity.

Plugin information should naturally enter the existing World.

It should never create:

- separate workflows
- separate navigation
- separate interaction models

The platform owns continuity.

Plugins participate within it.

Conceptual Model

[mermaid]
flowchart TD

```
PreviousWorld
PreviousWorld --> UserIntent
UserIntent --> UpdatedFocus
UpdatedFocus --> UpdatedContext
UpdatedContext --> UpdatedComposition
UpdatedComposition --> UpdatedWorld
```

Nothing is discarded.

Understanding accumulates.

Design Consequences

Prioritising continuity results in:

- calmer interfaces
- stronger trust
- reduced cognitive effort
- better long-term learnability
- fewer unnecessary transitions

Continuity should therefore be considered one of the defining behavioural characteristics of Mosaic.

Summary

Continuity is the preservation of understanding.

Users should feel they remain inside one evolving World rather than repeatedly travelling between disconnected interfaces.

When continuity succeeds...

Navigation becomes less important.

Understanding becomes effortless.

Review Status

Status

Draft

Next File

03-focus-transitions.md

Focus Transitions

Purpose

Focus is the centre of every Mosaic experience.

Consequently, changing Focus is one of the most important interactions within the platform.

This chapter defines how Focus should evolve.

It intentionally avoids discussing animation.

Instead, it defines the behavioural expectations that every implementation should communicate.

Definition

A **Focus Transition** is defined as:

The behavioural process by which the user's primary attention moves from one subject to another.

A Focus Transition does not necessarily imply navigation.

It simply indicates that something else has become the centre of the user's current World.

Why Focus Transitions Exist

Traditional applications frequently interpret every selection as navigation.

Select



New Page

Mosaic instead interprets selection as changing importance.

Select



Focus Changes



World Reorganises

The distinction is subtle.

It fundamentally changes how the platform behaves.

Focus Never Teleports

One of the fundamental behavioural rules of Mosaic is:

Focus should never appear to teleport.

Users should always understand:

- what was previously important
- what is becoming important
- why the change occurred

The transition should preserve understanding.

Not merely update interface.

Behavioural Sequence

Every Focus Transition should conceptually follow the same sequence.

[text]

Current Focus

↓

User Intent

↓

Focus Validation

↓

Context Update

↓

Composition Re-evaluation

↓

Presentation Update

Every later interaction model builds upon this sequence.

Explicit Focus

Some transitions are initiated directly by the user.

Examples include:

- selecting a film
- opening a book
- choosing another series
- beginning playback

These transitions possess the strongest confidence.

The platform should respond immediately.

Implicit Focus

Other transitions occur naturally.

Examples include:

- playback completes
- reading reaches the next chapter
- music playlist advances
- the next episode begins automatically

The platform should update Focus only when the user's intent remains clear.

Implicit transitions should never surprise the user.

Interrupted Focus

Users frequently interrupt themselves.

Examples include:

Watching

↓

Pause

↓

Browse Cast

↓

Resume Watching

The Focus has not fundamentally changed.

The user remains inside the same entertainment experience.

Temporary exploration should therefore preserve the original Focus wherever practical.

Browsing Is Not Focus

One of the most important distinctions within Mosaic is:

Browsing does not necessarily indicate Focus.

Example.

Watching Frieren

↓

Browse Breaking Bad

↓

Return

The platform should avoid assuming that simply opening another item permanently changes the user's Focus. Intent matters more than clicks.

Focus Confidence

Future implementations should associate every Focus with a confidence level.

Example.

Behaviour	Confidence
Active playback	Very High
Active reading	Very High
Explicit selection	High
Browsing metadata	Medium
Hovering artwork	Low
Passive recommendation exposure	None

The platform should avoid changing Focus based on weak signals.

This protects continuity.

Same-Domain Transitions

Changing Focus within the same Domain should feel evolutionary.

Example.

Anime

↓

Frieren

↓

Fire Force

Shared concepts remain.

Examples include:

- relationships
- timeline
- artwork hierarchy
- interaction model

The World evolves gently.

Cross-Domain Transitions

Changing Focus between Domains represents a more significant conceptual shift.

Example.

Anime



Books

Different information becomes important.

Different Expressions may become active.

Composition changes more substantially.

Despite this...

The user should still feel they remain inside the same World.

Returning Focus

Returning to a previous Focus should feel familiar.

The platform should preserve meaningful state.

Examples include:

- playback position
- reading progress
- expanded relationships
- current composition

Returning should never feel like beginning again.

Losing Focus

Sometimes the platform genuinely has no meaningful Focus.

Examples include:

- first launch
- empty library
- onboarding

During these situations the platform should establish a Focus naturally.

Examples include:

- onboarding flow
- current collection
- recent activity

The absence of Focus should never produce an empty experience.

Good Examples

Example 01

Current Focus

Frieren

User selects:

Episode 15

The Focus evolves naturally.

Everything continues to relate to Frieren.

Example 02

Current Focus

Reading Book

User opens:

Author Profile

The platform should understand that this is supporting exploration.

Not abandonment of the book.

Example 03

Playback completes.

The platform naturally transitions Focus towards:

Next Episode

The World continues.

Nothing resets.

Anti-patterns

Every Click Changes Focus

Treating every interaction as a new primary activity.

Users rapidly lose continuity.

Focus Resets

Completing playback immediately returns the user to an unrelated homepage.

The current journey is abandoned.

Predictive Focus

Popularity or recommendations replace the user's existing Focus.

The platform begins directing attention instead of supporting it.

Forgotten Focus

Returning to Mosaic after a short absence forces users to rediscover where they were.

The platform has failed to preserve continuity.

Relationship To Composition

Focus does not directly determine presentation.

Instead:

[mermaid]
flowchart TD

Focus
Focus --> Context
Context --> Composition
Composition --> Expressions
Expressions --> Presentation

Changing Focus influences every subsequent stage.

Presentation simply communicates the resulting changes.

Summary

Focus represents the current centre of the user's entertainment World.

Transitions should feel like shifts in attention.

Not changes of application.

Users should always understand:

- what changed
- why it changed
- how it relates to what came before

When Focus behaves consistently, the rest of Mosaic naturally becomes easier to understand.

Review Status

Status

Draft

Next File

04-context-transitions.md

Context Transitions

Purpose

While **Focus** defines what currently matters, **Context** explains why it matters.

A Focus Transition changes the subject of attention.

A Context Transition changes the user's relationship with that subject.

This distinction is fundamental to the Mosaic Interaction Model.

The same Focus may produce many different Contexts without ever requiring the user to leave their World.

Definition

A **Context Transition** is defined as:

A behavioural change that alters the relevance of information without necessarily changing the user's Focus.

Context changes continuously.

Focus changes relatively infrequently.

The Interaction Model therefore expects Context Transitions to occur far more often than Focus Transitions.

Why Context Transitions Exist

Entertainment is rarely a single activity.

Consider the following sequence.

Watching Frieren

↓

Pause

↓

Browse Characters

↓

Read Episode Discussion

↓

Resume Playback

The Focus remains:

Frieren

However, the Context evolves repeatedly.

Treating each Context as navigation would fragment the experience.

Instead, Mosaic understands that the user is simply exploring different aspects of the same World.

Behavioural Sequence

Every Context Transition follows the same conceptual sequence.

[text]

Current Context

↓

User Intent

↓

Relevant Information Changes

↓

Composition Re-evaluates

↓

Presentation Adapts

Notice that Focus remains unchanged.

Context Is Layered

Multiple Contexts may coexist.

Example.

World

↓

Anime

↓

Frieren

↓

Episode 14

↓

Watching

↓

Paused

Each layer provides additional understanding.

The deepest Context generally has the strongest influence over the current Composition.

Context Evolves Continuously

Context naturally changes during interaction.

Examples include:

Browsing

↓

Watching

↓

Paused

↓

Watching

↓

Completed

No navigation is required.

The World simply evolves.

Temporary Contexts

Some Contexts exist only briefly.

Examples include:

- searching
- filtering
- comparing
- sharing
- viewing metadata

Temporary Contexts should never permanently replace the user's underlying activity.

Once completed, the previous Context should naturally resume.

Persistent Contexts

Other Contexts may remain active for extended periods.

Examples include:

- binge watching
- reading a long novel
- listening to an album
- exploring a franchise

Persistent Contexts should accumulate understanding over time.

The platform should become increasingly helpful rather than repeatedly restarting the experience.

Context Changes Priority

Context directly influences informational priority.

Example.

Current Context

Watching

Priority becomes:

- playback
- subtitles
- next episode

Changing Context.

Browsing Characters

Priority naturally shifts towards:

- cast
- relationships
- voice actors
- appearances

Nothing about the World has changed.

Only relevance has.

Context Does Not Reset Composition

One of the core behavioural rules of Mosaic is:

Context should evolve the existing Composition whenever practical.

Poor behaviour.

Watching

↓

Character List

↓

Completely New Interface

Preferred behaviour.

Watching

↓

Characters Become Relevant

↓

Composition Evolves

The user remains oriented.

Context Informs Discovery

Discovery should emerge from Context.

Example.

Current Context.

Reading Dune

Useful discoveries include:

- sequels
- audiobook
- Frank Herbert
- film adaptations

The platform is not recommending unrelated books.

It is deepening the current Context.

Context Expiration

Every Context possesses a natural lifetime.

Examples.

Paused

↓

Expires

↓

Watching

or

Searching

↓

Search Complete

↓

Previous Context Restored

Context should rarely linger longer than necessary.

Expired Context should gracefully return control to the broader World.

Context Across Devices

Context should persist across platforms.

Example.

Reading

↓

Tablet

↓

Continue

↓

Phone

The device changes.

The Context remains.

Users should never need to manually reconstruct what they were doing.

Good Examples

Example 01

Current Context

Watching Episode

User opens:

Subtitles

Playback remains central.

The subtitle interface appears temporarily.

Closing subtitles naturally restores the previous Context.

Example 02

Current Context

Reading Book

User views:

Character Glossary

Reading remains the dominant activity.

The glossary enriches rather than replaces the experience.

Example 03

Current Context

Series Overview

User begins playback.

The Context naturally becomes:

Watching Episode

Composition adapts accordingly.

Anti-patterns

Full Navigation

Every Context Transition behaves like opening another application.

The World fragments.

Context Loss

Returning from supporting information unexpectedly resets the user's current activity.

Overreaction

Minor Context changes trigger complete recomposition.

Users lose continuity.

Persistent Temporary States

Temporary Contexts remain active after they are no longer useful.

The platform appears confused.

Context Resolution

Future runtime systems should determine Context using multiple signals.

Potential inputs include:

- active playback
- explicit user actions
- elapsed time
- interaction history
- active device
- available media

MDL intentionally avoids prescribing the implementation.

It prescribes only the expected behaviour.

Relationship To Composition

Context determines informational relevance.

Composition determines how that relevance is communicated.

[mermaid]
flowchart TD

```
Context
Context --> RelevantInformation["Relevant Information"]
RelevantInformation["Relevant Information"] --> Priority
Priority --> Composition
Composition --> Expressions
Expressions --> Presentation
```

Changing Context should therefore influence composition naturally rather than forcing abrupt interface replacement.

Summary

Context Transitions are the most common behavioural changes within Mosaic.

Unlike Focus Transitions, they rarely change what the user cares about.

Instead, they change:

- what information is relevant
- what deserves emphasis
- what should quietly move into the background

The World remains.

The Focus usually remains.

Only understanding evolves.

Review Status

Status

Draft

Next File

05-composition-evolution.md

Composition Evolution

Purpose

Composition is not static.

It is a living representation of the user's current World.

As the World evolves, the Composition should evolve with it.

This chapter defines how compositions change over time without fragmenting the user's understanding.

Unlike traditional page-based applications, Mosaic does not replace compositions.

It continuously refines them.

Definition

A **Composition Evolution** is defined as:

The continuous reorganisation of a Composition as the user's World changes.

Composition evolution exists to preserve understanding while reflecting new information.

The objective is not visual novelty.

The objective is continuity.

Why Composition Evolves

The user's World is constantly changing.

Examples include:

- playback progressing
- an episode completing
- a chapter finishing
- an album ending
- new information arriving
- plugins contributing knowledge
- relationships becoming relevant

If Composition remained static, the interface would rapidly become less useful.

If Composition changed too aggressively, users would lose orientation.

Composition Evolution exists to balance these two extremes.

The Behavioural Model

Composition should evolve using the following sequence.

[text]

World Changes

↓

Information Changes

↓

Relationship Importance Changes

↓

Composition Re-evaluates

↓

Expressions Re-evaluate

↓

Presentation Updates

Notice that Presentation remains the final concern.

Composition evolves conceptually before anything visual changes.

Evolution Is Continuous

Compositions should evolve gradually.

Poor.

Old Composition

↓

New Composition

Preferred.

Old Composition

↓

Information Changes

↓

Priority Changes

↓

Composition Evolves

↓

Understanding Preserved

The user should rarely feel that an interface has been replaced.

Evolution Is Local

Whenever practical, changes should remain local.

Example.

Playback reaches 95%.

Only:

- progress
- remaining runtime
- next episode

need additional emphasis.

The entire interface should not reorganise simply because one value changed.

Local evolution preserves stability.

Progressive Evolution

Large behavioural changes should occur through multiple small evolutions.

Example.

Episode Ends.

Instead of:

Player Closes

↓

Everything Changes

Prefer.

Playback Ends

↓

Progress Completes

↓

Timeline Expands

↓

Next Episode Gains Priority

↓

Playback Controls Reduce

↓

Composition Settles

Each individual change is understandable.

Together they communicate a complete behavioural transition.

Stable Anchors

Every Composition should contain stable anchors.

Anchors provide orientation while other elements evolve.

Examples include:

- Domain Navigation
- Search
- Current Focus
- Global Playback

Anchors should move rarely.

Their stability allows the rest of the interface to remain adaptive without becoming disorienting.

Adaptive Elements

Other parts of the Composition are intentionally adaptive.

Examples include:

- related information
- recommendations
- timeline
- metadata
- extension contributions

These elements should respond naturally to changing Context.

The distinction between Anchors and Adaptive Elements allows Mosaic to feel dynamic without becoming unstable.

Composition Density

Evolution may change information density.

Example.

Watching.

Sparse

↓

High Focus

Browsing.

Denser

↓

Exploration

The Composition should adapt according to user intent rather than arbitrary screen templates.

Density is therefore an output of Context.

Not a predefined layout.

Composition Compression

When available space becomes limited:

The Composition should compress gracefully.

Compression should occur in the following order.

1. Low Priority Information 2. Supporting Relationships 3. Secondary Metadata 4. Decorative Information

Core understanding should always remain visible.

Compression should never remove the user's ability to understand their current World.

Composition Expansion

As space or relevance increases:

The Composition should expand naturally.

Expansion should reveal:

- additional relationships
- richer metadata
- deeper exploration
- broader context

Expansion should never introduce unrelated information simply because additional space exists.

Evolution Triggers

The following events should normally trigger Composition Evolution.

User Driven

- selecting media
- beginning playback
- finishing playback
- searching
- filtering
- changing Domain

System Driven

- episode released
- metadata updated
- plugin contribution
- relationship discovered
- download completed

Temporal

- countdown completed
- scheduled release
- reading session resumed
- long inactivity

Every trigger should produce understandable behavioural evolution.

Composition Memory

A Composition should remember meaningful user state.

Examples include:

- expanded sections
- current Focus
- reading position
- playback position
- selected relationships

Users should rarely feel they are repeatedly reconstructing the same interface.

Composition should accumulate understanding rather than continually forgetting it.

Plugin Behaviour

Plugins should never directly manipulate Composition.

Instead they contribute:

- Information
- Relationships

The Composition Engine decides:

- importance
- priority

- grouping
- emphasis

This preserves one coherent behavioural model regardless of installed extensions.

Good Examples

Example 01

Airing Episode

Tomorrow

↓

Today

↓

Now Available

The Timeline naturally gains priority.

The rest of the Composition remains stable.

Example 02

Book Finished.

Progress completes.

Series continuation becomes more important.

Author information expands.

The World continues naturally.

Example 03

Music Album Ends.

Playback controls reduce.

Suggested continuation appears.

Current artist remains the conceptual centre.

The Composition evolves rather than restarting.

Anti-patterns

Complete Recomposition

Every update rebuilds the entire interface.

Users lose orientation.

Static Composition

Information never adapts regardless of changing Context.

The interface becomes progressively less useful.

Priority Inversion

Low-value information unexpectedly dominates the Composition.

Understanding decreases.

Plugin Recomposition

Extensions rearrange interface independently of the Composition Engine.

The behavioural model fragments.

Composition Evolution Model

[mermaid]
flowchart TD

```
World
World --> InformationChanges["Information Changes"]
InformationChanges["Information Changes"] --> RelationshipChanges["Relationship Changes"]
RelationshipChanges["Relationship Changes"] --> PriorityChanges["Priority Changes"]
PriorityChanges["Priority Changes"] --> CompositionEvolves["Composition Evolves"]
CompositionEvolves["Composition Evolves"] --> ExpressionsUpdate["Expressions Update"]
ExpressionsUpdate["Expressions Update"] --> PresentationUpdates["Presentation Updates"]
```

This sequence should remain consistent throughout every Mosaic client.

Summary

Composition Evolution allows Mosaic to remain alive without becoming unpredictable.

The World evolves.

The Composition reflects that evolution.

Users should never feel that they are repeatedly entering new interfaces.

Instead, they should feel that their entertainment World quietly reorganises itself around what now matters.

Review Status

Status

Draft

Next File

06-movement.md

Movement

Purpose

Movement is the physical manifestation of behavioural change.

It exists to communicate the evolution of the user's World.

Within the Mosaic Design Language, movement is not decoration.

It is explanation.

If Composition determines **what changed**, movement communicates **how that change occurred**.

Definition

Within MDL, **Movement** is defined as:

The visual communication of behavioural change while preserving the user's understanding of their World.

Movement is a communication system.

It is not an animation system.

Animation is an implementation.

Movement is intent.

Why Movement Exists

Every interaction changes something.

Examples include:

- Focus changes
- Context changes
- Relationships become important
- Information becomes available
- Composition evolves

Without movement the interface appears to change instantaneously.

Users must reconstruct understanding from scratch.

Movement preserves continuity by explaining:

- what changed
- where it changed
- why it changed

Behaviour Before Motion

Movement should always communicate behaviour that already exists.

Incorrect process.

Animation

↓

Behaviour

Correct process.

Behaviour

↓

Movement

↓

Animation

Animation implements movement.

Movement implements behaviour.

Behaviour implements the Mental Model.

The Purpose Of Movement

Movement exists for four reasons.

Preserve Continuity

Users should understand where things came from.

Explain Change

Users should understand why something changed.

Reinforce Hierarchy

Important changes deserve greater emphasis than insignificant ones.

Reduce Cognitive Effort

Movement should reduce interpretation rather than increase it.

Movement Is Directional

Movement should always possess direction.

Direction communicates relationship.

Examples.

Focus

↓

Expands

Supporting Information

↓

Moves Aside

Completed Information

↓

Leaves Composition

Direction provides meaning.

Random movement provides none.

Movement Is Intentional

Every visible movement should answer a question.

Examples.

Question.

Why did this move?

Answer.

Focus changed.

Question.

Why did this disappear?

Answer.

It is no longer relevant.

Question.

Why did this appear?

Answer.

New information became important.

If no meaningful answer exists...

The movement should not exist.

Conceptual Weight

Movement should reflect conceptual weight rather than visual distance.

Example.

Episode 14

↓

Episode 15

Small conceptual change.

Small movement.

Example.

Anime

↓

Books

Large conceptual change.

Larger movement.

Example.

Playback

↓

Administration

Very large conceptual change.

The platform should clearly communicate that the user has entered a fundamentally different mode.

Stable Objects

Objects should preserve identity wherever practical.

Users should recognise:

Current Progress

↓

Current Progress

rather than

Old Progress

↓

Destroyed

↓

New Progress

The object remains.

Only its state changes.

This greatly improves long-term learnability.

Local Movement

Movement should normally remain local.

Changing:

Progress

should not cause:

- Search
- Navigation
- Hero
- Timeline

to animate unnecessarily.

Only objects participating in the behavioural change should move.

Local movement reduces visual noise.

Hierarchical Movement

Movement should follow hierarchy.

Examples.

Primary Focus moves first.

Supporting information follows.

Background information moves last.

This naturally reinforces importance without additional interface elements.

Arrival

New information should arrive naturally.

Poor.

Information

↓

Instantly Appears

Preferred.

Information Becomes Relevant

↓

Composition Creates Space

↓

Information Joins

Users should understand why something has entered their World.

Departure

Likewise...

Information should leave naturally.

Poor.

Information

↓

Disappears

Preferred.

Information Becomes Less Relevant

↓

Priority Reduces

↓

Leaves Composition

Departure should feel like completion.

Not deletion.

Movement And Time

Movement should communicate temporal progression.

Examples.

Watching

↓

Completed

↓

Continue

↓

Next Episode

The interface should communicate that one experience has naturally progressed into another.

Users should never feel abruptly returned to the beginning.

Plugin Behaviour

Plugins never define movement.

Plugins contribute:

- Information
- Relationships

The platform determines:

- arrival
- departure
- emphasis
- continuity

This ensures one coherent behavioural language regardless of installed extensions.

Good Examples

Example 01

Episode completes.

Progress fills.

Timeline advances.

Next episode gains emphasis.

Playback controls reduce.

The World continues.

Example 02

User opens Cast.

Character information expands naturally around the current Focus.

Playback remains recognisable.

The user understands they are exploring rather than navigating away.

Example 03

New episode releases overnight.

The next morning.

Timeline quietly gains emphasis.

Nothing else unnecessarily changes.

The World has evolved.

Anti-patterns

Decorative Motion

Movement exists because it looks impressive.

Understanding does not improve.

Teleportation

Objects disappear.

New objects appear elsewhere.

Identity is lost.

Simultaneous Motion

Everything moves together.

Hierarchy disappears.

Users no longer know where to look.

Performance Before Understanding

Movement is removed simply because it is technically expensive.

Understanding decreases.

Optimisation should preserve behavioural meaning wherever possible.

Behavioural Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> Meaning
Meaning --> Movement
Movement --> Animation
Animation --> Presentation
```

Animation is the final implementation detail.

Behaviour always remains the primary concern.

Summary

Movement is the behavioural language of Mosaic.

It communicates:

- continuity

- hierarchy
- evolution
- understanding

Movement should never exist because animation is possible.

It exists because users deserve to understand how their World has changed.

Review Status

Status

Draft

Next File

07-user-flow.md

User Flow

Purpose

Traditional software describes users as moving through interfaces.

The Mosaic Interaction Model intentionally rejects this perspective.

Users do not move through Mosaic.

They move through **their entertainment**.

The platform simply follows.

This chapter defines how user flow should be understood throughout the Mosaic Design Language.

Definition

Within MDL, **User Flow** is defined as:

The continuous progression of a user's intent through their entertainment World.

Notice that the definition does not mention:

- pages
- screens
- routes
- navigation

Those are implementation concerns.

The flow being modelled is the user's thinking.

Not the application's structure.

Why User Flow Exists

People rarely describe their evening like this.

Open Homepage

↓

Navigate Library

↓

Open Series

↓

Open Season

↓

Play Episode

Instead they describe it like this.

Continue Frieren.

or

Read another chapter.

or

Listen to that soundtrack again.

The software journey is invisible.

Only the entertainment journey matters.

Mosaic therefore models the latter.

Intent

Every user flow begins with intent.

Intent answers:

What is the user trying to achieve?

Examples include:

- Continue watching
- Continue reading
- Discover something related
- Resume listening
- Explore a franchise
- Manage a collection

Everything else exists to support that intent.

The Flow Model

Every interaction should conceptually follow the same behavioural flow.

[text]
Intent

↓

World

↓

Focus

↓

Context

↓

Composition

↓

Understanding

↓

Entertainment

Notice that the user never "arrives" at a page.

They arrive at understanding.

User Goals

Every user flow should optimise one of four goals.

Continue

Resume an existing experience.

Examples:

- Continue Watching
- Continue Reading
- Resume Playback

This should represent the lowest-friction flow in the platform.

Explore

Deepen the current experience.

Examples:

- Cast
- Author
- Soundtrack
- Manga
- Timeline

Exploration should remain closely connected to the current Focus.

Discover

Find something new.

Discovery should emerge naturally from:

- relationships
- current interests
- current Context

Discovery should never interrupt ongoing entertainment.

Manage

Perform administration.

Examples include:

- downloads
- users
- plugins
- libraries

Management intentionally behaves differently from entertainment.

It should remain calm, structured and predictable.

Flow Types

The Interaction Model recognises four behavioural flow types.

Flow	Behaviour
Continue	Lowest friction
Explore	Expand understanding
Discover	Widen the World
Manage	Configure the platform

The interface should always understand which flow the user is currently following.

Flow Continuity

A user should be capable of leaving a flow...

...and naturally returning.

Example.

Watching

↓

Pause

↓

Browse Cast

↓

Resume Watching

The browsing flow should never destroy the playback flow.

Instead it should temporarily branch from it.

Branching

User flow should branch naturally.

[mermaid]
flowchart TD

```
ContinueWatching["Continue Watching"]
ContinueWatching["Continue Watching"] --> Episode
Episode --> Characters
Episode --> Soundtrack
Episode --> Manga
Characters --> Episode
Soundtrack --> Episode
Manga --> Episode
```

Notice that every branch naturally returns to the current entertainment experience.

This prevents fragmentation.

Dead Ends

Dead ends should be considered design failures.

Example.

Open Actor

↓

Nothing Else

The user's journey stops.

Instead:

Actor

↓

Other Films

↓

Related Directors

↓

Current Focus

Every flow should naturally lead somewhere meaningful.

Flow Memory

Mosaic should remember meaningful user flow.

Examples include:

- last reading position

- current episode
- expanded relationships
- current search
- unfinished exploration

Returning users should feel as though they have resumed a conversation rather than started a new one.

Friction

Every transition within a user flow introduces friction.

Examples include:

- additional clicks
- unnecessary confirmations
- repeated searches
- repeated navigation
- repeated context switching

The objective of Mosaic is to remove as much of this friction as possible without removing user understanding.

Flow Priority

When multiple flows are possible, they should normally be prioritised in the following order.

1. Continue 2. Explore 3. Discover 4. Manage

This ordering reflects the philosophy established in MDL-001.

Helping users continue what they already chose is generally more valuable than encouraging unrelated discovery.

Good Examples

Continue

Open Mosaic

↓

Continue Watching

↓

Playback

Minimal friction.

Explore

Current Episode



Voice Actor



Other Roles



Return

The exploration remains connected to the original experience.

Discover

Current Book



Related Author



Another Series

Discovery grows naturally from existing interest.

Anti-patterns

Forced Navigation

The user repeatedly returns to a homepage before continuing.

Flow Reset

Small interruptions completely restart the entertainment journey.

Promotional Diversion

The platform redirects the user away from their current intent.

Management Dominance

Administrative workflows become more visible than entertainment workflows.

Relationship To Other Specifications

User Flow builds directly upon:

- World
- Focus
- Context
- Composition

It intentionally avoids introducing new concepts.

Instead, it explains how existing concepts work together over time.

Summary

Users do not flow through interfaces.

They flow through experiences.

Mosaic should therefore optimise the journey through entertainment rather than the journey through software.

When contributors discuss user flow, they should always begin by asking:

What is the user trying to continue?

Not:

Which screen should they visit next?

Review Status

Status

Draft

Next File

08-temporal-behaviour.md

Temporal Behaviour

Purpose

The previous chapters described how the user's World evolves in response to interaction.

This chapter introduces a second dimension.

Time.

Unlike traditional applications, Mosaic should not only react to user input.

It should also react to the natural passage of time.

Time is considered a first-class behavioural input.

The user's World changes even while they are not actively interacting with the application.

The Interaction Model must therefore define how Mosaic behaves before, during and after those changes occur.

Definition

Within MDL, **Temporal Behaviour** is defined as:

The behavioural changes produced by the passage of time rather than direct user interaction.

Examples include:

- episode releases
- countdown completion
- scheduled events
- reading inactivity
- playback completion
- background metadata updates

Temporal behaviour allows Mosaic to remain alive without becoming intrusive.

Why Time Matters

Entertainment naturally unfolds through time.

Examples include:

- episodes air
- books are completed
- albums release
- concerts are announced

- downloads finish
- subtitles become available

Traditional applications generally wait for users to discover these changes.

Mosaic should instead quietly integrate them into the user's World.

The World should appear to evolve naturally.

Time Is Not A Notification

One of the most important distinctions within MDL is that temporal behaviour is **not** equivalent to notifications.

Example.

Poor behaviour.

Episode Released

↓

Notification

↓

Dismiss

Preferred behaviour.

Episode Released

↓

Timeline Evolves

↓

Continue Watching Gains Priority

↓

World Updates

The user discovers the change naturally.

Not because the platform demanded attention.

Temporal States

Every meaningful piece of Information exists within time.

Examples include:

Future

↓

Present

↓

Past

Example.

Episode

↓

Announced

↓

Airing Tomorrow

↓

Available

↓

Watched

↓

Completed

The Information remains identical.

Its temporal state evolves.

Behaviour Before Availability

Before something becomes available, Mosaic should reduce uncertainty.

Example.

Next Episode

↓

Tomorrow

Questions naturally answered include:

- When?
- How long?
- What happens next?

The platform should reduce anxiety caused by uncertainty rather than merely reporting availability after the fact.

Behaviour At Availability

When Information becomes available:

The World should evolve naturally.

Examples include:

- Timeline updates
- Continue Watching changes

- Progress resumes
- Composition re-evaluates

The objective is not to celebrate the event.

The objective is to integrate it.

Behaviour After Completion

Completion should never feel like termination.

Poor.

Episode Ends

↓

Finished

Preferred.

Episode Ends

↓

Progress Updates

↓

Relationships Expand

↓

Next Episode Gains Priority

↓

World Continues

Completion represents continuity.

Not conclusion.

Passive Evolution

Many changes should occur without requiring user interaction.

Examples include:

- new episode releases
- metadata refresh
- artwork updates
- relationship discovery
- plugin synchronisation

Users should simply discover a richer World the next time they return.

The platform should avoid unnecessarily exposing background activity.

Active Evolution

Some temporal changes occur while the user is actively engaged.

Examples include:

- playback reaches credits
- chapter completes
- countdown expires
- download finishes

These changes should be incorporated smoothly into the existing experience.

The platform should never appear startled by events it could reasonably anticipate.

Inactivity

Time also affects inactivity.

After extended absence:

The platform should assume continuity rather than abandonment.

Example.

User returns after six months.

Instead of presenting:

- every change
- every recommendation
- every update

Mosaic should begin with the user's previous World.

Only after the user's Focus changes should additional areas of the World become emphasised.

The platform resumes the conversation.

It does not begin a new one.

Temporal Confidence

Future runtime systems may associate confidence with temporal events.

Examples.

Event	Confidence
Scheduled Episode Release	Very High
Playback Completion	Very High
Metadata Refresh	High

Event	Confidence
Community Update	Medium
External Recommendation	Low

Higher-confidence events should produce stronger behavioural adaptation.

Lower-confidence events should remain peripheral.

Temporal Rhythm

The platform should develop a natural rhythm.

Preparation

↓

Availability

↓

Continuation

↓

Completion

↓

Preparation

This rhythm should become recognisable across every entertainment domain.

Television.

Books.

Music.

Films.

The behavioural pattern remains consistent.

Only the Information changes.

Plugins

Plugins should contribute temporal Information.

Examples.

Anime Plugin.

Episode

↓

Tomorrow

Music Plugin.

Concert

↓

Next Month

Book Plugin.

Next Volume

↓

Publishing Soon

The plugin contributes temporal knowledge.

The platform decides:

- timing
- emphasis
- composition
- presentation

Anti-patterns

Notification First

Time always produces alerts.

The interface becomes demanding.

Time Ignorance

The World never evolves until manually refreshed.

The platform feels static.

Temporal Reset

Users returning after absence lose continuity.

The platform behaves as though it has forgotten them.

Information Flood

Every historical change is presented simultaneously.

Users become overwhelmed.

Temporal Behaviour Model

[mermaid]
flowchart TD

```
Time
Time --> InformationChanges["Information Changes"]
InformationChanges["Information Changes"] --> RelationshipChanges["Relationship Changes"]
RelationshipChanges["Relationship Changes"] --> CompositionReevaluates["Composition Re-evaluates"]
CompositionReevaluates["Composition Re-evaluates"] --> ExpressionsUpdate["Expressions Update"]
ExpressionsUpdate["Expressions Update"] --> PresentationEvolves["Presentation Evolves"]
```

Time should influence understanding.

Not merely trigger interface updates.

Summary

Time is one of the primary behavioural inputs to Mosaic.

The platform should quietly evolve alongside the user's entertainment.

Users should never feel they must constantly monitor the platform.

Instead, they should trust that their World will naturally become richer with time.

Review Status

Status

Draft

Next File

09-interaction-states.md

Interaction States

Purpose

Every interaction within Mosaic exists within a behavioural state.

Unlike traditional applications, where state is frequently represented by interface elements such as pages, dialogs or navigation, Mosaic models state as part of the user's evolving World.

Interaction States define **what the user is currently trying to accomplish**.

They do not define **what interface they are looking at**.

Understanding this distinction is essential to maintaining behavioural continuity across every Mosaic client.

Definition

Within MDL, an **Interaction State** is defined as:

The current behavioural mode of the user's World.

Interaction States describe intent.

They do not describe interface.

Examples include:

- Watching
- Reading
- Listening
- Exploring
- Searching
- Managing

The same interface may represent multiple Interaction States over time.

Likewise, the same Interaction State may appear differently on different devices.

Why Interaction States Exist

Traditional software frequently models state like this.

Home

↓

Series Page

↓

Episode Page

↓

Player

Each state corresponds directly to interface.

Mosaic intentionally models behaviour instead.

Exploring

↓

Watching

↓

Paused

↓

Continuing

The interface merely communicates those behavioural states.

Behaviour Before Interface

Interaction States should always describe behaviour.

Never implementation.

Poor examples.

- Dialog Open
- Drawer Expanded
- Sidebar Hidden

Good examples.

- Exploring
- Watching
- Reading
- Comparing
- Continuing

The first describes software.

The second describes people.

Primary States

The following Interaction States are considered foundational.

Exploring

Purpose:

Understanding possibilities.

Typical behaviours:

- browsing
- comparing
- learning
- discovering

Exploration should expand understanding without abandoning the current World.

Watching

Purpose:

Consuming video.

Primary emphasis:

- playback
- progress
- next episode

Everything unrelated should quietly reduce emphasis.

Reading

Purpose:

Consuming written content.

Primary emphasis:

- chapter
- reading position
- bookmarks
- continuation

The interface should become significantly quieter.

Listening

Purpose:

Consuming audio.

Primary emphasis:

- playback
- queue
- progress
- currently playing

Artwork remains important.

Interface chrome should recede.

Continuing

Purpose:

Resuming an existing experience.

This is considered the lowest-friction state within Mosaic.

Users should require minimal conscious interaction before returning to entertainment.

Managing

Purpose:

Administration.

Examples include:

- plugins
- users
- libraries
- storage

Management intentionally follows different behavioural priorities.

Entertainment should remain adaptive.

Management should remain predictable.

Transitional States

Some states exist only briefly.

Examples include:

- Loading
- Synchronising
- Preparing Playback
- Searching
- Authenticating

These states should communicate progress.

They should never become destinations.

The objective is to move through them rather than remain within them.

Nested States

Interaction States may contain additional states.

Example.

Watching

■■■ Playing

■■■ Paused

■■■ Seeking

■■■ Completed

The parent state remains:

Watching

Nested states should not fundamentally alter the user's understanding of their World.

State Ownership

Every Interaction State owns:

- behavioural expectations
- information priority
- composition emphasis
- interaction affordances

Interaction States intentionally do **not** own:

- colours
- materials
- animation
- typography

Those belong to the Design System.

State Transitions

Interaction States should transition naturally.

Example.

Exploring

↓

Watching

↓

Paused

↓

Watching



Completed



Continuing

Every transition should reinforce continuity.

The user should understand why the state changed.

State Persistence

Some Interaction States should survive interruption.

Examples include:

Watching



Phone Call



Watching

or

Reading



Lock Screen



Reading

Temporary interruptions should not destroy the user's current journey.

State Restoration

Returning to Mosaic should restore meaningful behavioural state whenever practical.

Examples include:

- playback
- reading
- exploration
- searches
- filters

The user should feel as though the platform remembered the conversation.

Not restarted it.

State Priority

Only one primary Interaction State should normally exist at a time.

Secondary states may exist beneath it.

Example.

Watching

↓

Searching Subtitles

The dominant state remains:

Watching

This hierarchy prevents competing behavioural priorities.

Plugins

Plugins should never introduce entirely new primary Interaction States.

Instead they should enrich existing states.

Example.

A Manga extension should strengthen:

Reading

rather than creating:

Manga Mode

The behavioural language remains coherent across the ecosystem.

Anti-patterns

Interface States

Treating interface conditions as behavioural states.

Examples.

- Dialog Open
- Card Expanded
- Modal Visible

These describe implementation.

Not user behaviour.

Competing States

The interface simultaneously behaves as though the user is:

- Watching
- Managing
- Discovering
- Configuring

The platform loses behavioural clarity.

Forgotten State

Returning users begin from a generic homepage.

Their previous behavioural state has been discarded.

Continuity is broken.

Behaviour Model

```
[mermaid]
stateDiagram-v2
```

```
[*] --> Exploring
```

```
Exploring --> Watching
```

```
Exploring --> Reading
```

```
Exploring --> Listening
```

```
Watching --> Paused
```

```
Paused --> Watching
```

```
Watching --> Completed
```

```
Completed --> Continuing
```

```
Continuing --> Watching
```

```
Reading --> Continuing
```

```
Listening --> Continuing
```

States describe behaviour.

Presentation communicates those behaviours.

Summary

Interaction States define **what the user is doing**.

Not:

- what page they are on
- what component is visible
- what technology is active

The interface should continuously reinforce behavioural state while allowing implementation to evolve independently.

Interaction States therefore become one of the primary mechanisms through which Mosaic preserves continuity.

Review Status

Status

Draft

Next File

10-user-vs-system-behaviour.md

User Behaviour vs System Behaviour

Purpose

The Interaction Model distinguishes between two fundamentally different kinds of behaviour.

The first is **user behaviour**.

The second is **system behaviour**.

Confusing these two models is one of the most common causes of poor interaction design.

Systems frequently optimise their own behaviour rather than supporting the behaviour of the person using them.

Mosaic intentionally reverses this relationship.

The platform exists to support user behaviour.

It does not expect users to adapt to system behaviour.

Two Behavioural Models

Every interaction within Mosaic can be viewed from two perspectives.

[text]

User Behaviour

↓

System Behaviour

↓

Visible Experience

User behaviour describes intent.

System behaviour describes implementation.

Visible experience is simply the translation between the two.

User Behaviour

User behaviour answers questions such as:

- What am I trying to do?
- Why am I here?
- What do I expect next?
- What information do I need?

Examples include:

Continue watching.

Continue reading.

Find the next episode.

Explore this author.

Resume listening.

Notice that none of these describe software.

They describe intention.

System Behaviour

System behaviour answers questions such as:

- Which data changed?
- Which composition should update?
- Which information became relevant?
- Which relationships require recalculation?
- Which Expressions should be regenerated?

These behaviours belong entirely to the platform.

Users should rarely become aware they exist.

The Translation Layer

One of Mosaic's primary responsibilities is translating user behaviour into system behaviour.

```
[mermaid]
flowchart LR
```

```
Intent["User Intent"]
```

```
Behavior["System Behaviour"]
```

```
Experience["Visible Experience"]
```

```
Intent --> Behavior
```

```
Behavior --> Experience
```

Users should never need to understand the intermediate step.

Behavioural Ownership

User Behaviour owns:

- intention
- goals
- expectations
- understanding

System Behaviour owns:

- computation
- composition
- prioritisation
- rendering
- synchronisation

The platform should never expose System Behaviour simply because it is technically convenient.

Good Translation

Example.

User Behaviour.

Continue watching.

System Behaviour.

Restore Focus.

↓

Restore Context.

↓

Recalculate Composition.

↓

Resume Playback.

Visible Experience.

Episode resumes.

The user never sees the intermediate behaviour.

Poor Translation

User Behaviour.

Continue watching.

Visible Experience.

Metadata refreshing...

Provider synchronising...

Relationship graph rebuilding...

Loading plugins...

The platform has leaked implementation.

The user's intent has been interrupted by system concerns.

Behaviour Should Be Invisible

The best system behaviour is behaviour users never consciously notice.

For example.

A new episode releases overnight.

System Behaviour.

Information Updates

↓

Relationships Update

↓

Composition Updates

User Behaviour.

Open Mosaic.

↓

Continue Watching.

The platform quietly prepared the experience.

The user simply experiences continuity.

Behavioural Timing

Not every system behaviour should immediately become visible.

Good examples.

- metadata refresh
- artwork updates
- relationship indexing
- plugin synchronisation

These should generally occur in the background.

The visible experience should only change when those changes become meaningful to the current World.

Behavioural Interruptions

System behaviour should interrupt the user only when:

- immediate action is required
- data loss is possible
- security requires confirmation
- understanding would otherwise be reduced

Everything else should remain in the background.

This reflects the Companion philosophy established in MDL-001.

Behavioural Consistency

Different implementations may perform very different internal work.

Desktop.

Local Database

Television.

Streaming API

Mobile.

Offline Cache

Despite these differences...

The user should experience the same behaviour.

The Interaction Model therefore governs experience rather than implementation.

Plugins

Extensions should contribute capability.

They should never redefine behavioural expectations.

For example.

Anime Plugin.

System Behaviour.

Episode Released

Platform Behaviour.

Composition Updates

User Behaviour.

Continue Watching.

Every plugin should reinforce the same behavioural language.

Anti-patterns

System First

The platform exposes technical operations before supporting the user's intent.

Technical Language

Users encounter implementation concepts such as:

- cache
- provider
- refresh
- synchronisation

when attempting ordinary entertainment tasks.

Behaviour Fragmentation

Different plugins introduce different interaction expectations.

Users must continually relearn behaviour.

The Companion disappears.

Behavioural Surprises

The system performs visible changes without understandable user benefit.

Trust decreases.

Behavioural Priority

When system behaviour and user behaviour conflict:

User behaviour always takes priority.

Examples.

System wants:

Refresh metadata.

User wants:

Continue reading.

The platform should continue reading.

Metadata can wait.

Supporting entertainment always takes precedence over maintaining the platform.

Behaviour Model

[mermaid]
flowchart TD

```
UserIntent
UserIntent --> SystemBehaviour
SystemBehaviour --> Composition
Composition --> Presentation
Presentation --> Understanding
```

Notice that the user never interacts directly with the system model.

Only with understanding.

Summary

The Interaction Model intentionally separates:

- what the user is trying to achieve

from

- what the platform must do internally.

This separation allows Mosaic to remain:

- calm
- predictable
- trustworthy

while still supporting increasingly sophisticated engineering beneath the surface.

Users should remember their entertainment.

Not the behaviour of the software supporting it.

Review Status

Status

Draft

Next File

11-governance.md

Interaction Model Governance

Purpose

The Interaction Model defines how Mosaic behaves.

Unlike implementation, behaviour becomes part of the user's long-term memory.

Changing interaction is therefore significantly more expensive than changing presentation.

This chapter defines how behavioural changes should be proposed, reviewed and maintained throughout the lifetime of Mosaic.

The objective of governance is not to prevent evolution.

It is to ensure that behavioural evolution remains coherent.

Behaviour Is Product Identity

Users rarely remember exact colours.

They rarely remember typography.

They rarely remember spacing.

They almost always remember:

- how a product behaves
- whether it felt predictable
- whether it respected their attention
- whether it became easier over time

Behaviour is therefore considered a primary part of the Mosaic identity.

Changing behaviour should be treated with the same care as changing the product vision.

Behavioural Stability

Interaction behaviour should remain significantly more stable than implementation.

Expected lifetime.

Layer	Expected Lifetime
Rendering	Weeks to Months
Components	Months
Motion Implementation	Months
Interaction Behaviour	Years
Mental Model	Decades
Vision	Decades

A new framework should never require a new interaction model.

What Requires Review

The following changes require formal Interaction Model review.

- New interaction patterns.
- New behavioural states.
- Changes to Focus behaviour.
- Changes to Context behaviour.
- Changes to behavioural continuity.
- Changes to navigation philosophy.
- Changes to interaction hierarchy.

These changes affect every Mosaic client.

What Does Not Require Review

The following normally belong to MDS rather than MDL.

- Animation timing.
- Motion curves.
- Easing.
- Typography.
- Colour.
- Materials.
- Visual transitions.
- Responsive layouts.

These concern presentation.

Not behaviour.

Behavioural Review Questions

Every proposed behavioural change should answer the following questions.

Question One

Does this strengthen continuity?

Question Two

Does this preserve understanding?

Question Three

Does this reduce friction?

Question Four

Does this reinforce the Companion philosophy?

Question Five

Would users naturally predict this behaviour after using Mosaic for several weeks?

Behaviour should become increasingly predictable over time.

Behavioural Drift

One of the greatest long-term risks to Mosaic is behavioural drift.

Behavioural drift occurs when:

- different clients behave differently
- plugins introduce competing interaction models
- exceptions accumulate
- new features ignore established behaviour
- navigation begins replacing continuity

Behavioural drift is more damaging than visual inconsistency because it weakens user trust.

Platform Consistency

Every Mosaic client should reinforce the same behavioural language.

Examples include:

Desktop.

Mobile.

Television.

Tablet.

Future devices.

Presentation may differ.

Behaviour should remain recognisable.

Users should never need to relearn:

- Focus
- Context
- Composition
- Interaction States

because they changed device.

Extension Governance

Plugins participate in the Interaction Model.

They do not redefine it.

Extensions may contribute:

- Information
- Relationships
- Capabilities

They should never independently define:

- navigation
- behavioural hierarchy
- interaction states
- continuity rules
- movement philosophy

Behaviour belongs to the platform.

Behavioural Regression

Every significant behavioural change should be considered a regression risk.

Examples include:

- users becoming disoriented
- continuity weakening
- navigation increasing
- interaction becoming less predictable

Future behavioural testing should therefore evaluate:

- orientation
- continuity
- understanding

- cognitive effort

rather than visual correctness alone.

Behavioural Evolution

Behaviour should evolve through refinement.

Preferred.

Existing Behaviour

↓

Refined Behaviour

Avoid.

Existing Behaviour

↓

Replacement Behaviour

unless there is compelling evidence that the original model no longer supports the Vision.

Interaction Debt

Technical debt affects implementation.

Interaction debt affects understanding.

Examples include:

- inconsistent navigation
- competing behavioural models
- duplicated interaction patterns
- unnecessary confirmations
- unpredictable focus changes

Interaction debt should be treated as a first-class architectural concern.

Removing behavioural inconsistency often improves user experience more than introducing new functionality.

Governance Workflow

[mermaid]
flowchart TD

```
BehaviourProposal["Behaviour Proposal"]
BehaviourProposal["Behaviour Proposal"] --> ExistingModelReview["Existing Model Review"]
ExistingModelReview["Existing Model Review"] --> Refinement["Refinement?"]
Refinement["Refinement?"] -->|Yes| UpdateBehaviour["Update Behaviour"]
Refinement["Refinement?"] -->|No| DesignReview["Design Review"]
DesignReview["Design Review"] --> ADR
ADR --> MDLUpdate["MDL Update"]
MDLUpdate["MDL Update"] --> DownstreamSpecifications["Downstream Specifications"]
```

The preferred outcome is refinement.

Replacement should remain rare.

Success Criteria

The Interaction Model succeeds when:

- behaviour feels consistent across every client
- users instinctively predict platform behaviour
- plugins naturally inherit platform interaction
- contributors solve behavioural problems using existing concepts
- new interaction patterns become increasingly rare

The strongest interaction model is one users eventually stop consciously noticing.

Architectural Decisions

ADR	Decision
ADR-038	Behaviour is considered part of the long-term product identity.
ADR-039	Behavioural consistency has higher priority than visual consistency.
ADR-040	Plugins participate in platform behaviour rather than defining independent interaction models.
ADR-041	Behaviour should evolve through refinement rather than replacement.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDL-004 document the significant behavioural decisions that define how Mosaic behaves over time.

Unlike the ADRs contained within:

- MDL-001 (Vision)
- MDL-002 (Principles)
- MDL-003 (Mental Model)

these decisions describe **behaviour**.

They explain why Mosaic behaves differently from traditional media applications.

Future contributors should use these ADRs to understand the behavioural intent behind the Interaction Model before proposing alternative interaction patterns.

Architecture Decision Records are intended to capture significant architectural decisions together with their context and consequences so future contributors understand not only *what* was decided but *why*.

[oai_citation:0±Google Cloud Documentation](https://docs.cloud.google.com/architecture/architecture-decision-records?hl=en&utm_source=chatgpt.com)

ADR Format

Every ADR within MDL follows the standard Mosaic format.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR should capture exactly one behavioural decision.

ADR-058

Title

Model Interaction As World Evolution

Status

Accepted

Context

Traditional applications generally model interaction as navigation between independent destinations.

Founder discovery consistently described entertainment as a continuous experience rather than a sequence of pages.

Decision

Interaction is defined as the continuous evolution of the user's World.

Navigation becomes one implementation of behavioural change rather than the primary behavioural model.

Consequences

Future interaction systems should prioritise continuity over relocation.

ADR-059

Title

Preserve Continuity Above Navigation

Status

Accepted

Context

Users rapidly construct mental maps while using software.

Frequent page replacement forces those maps to be rebuilt repeatedly.

Decision

Continuity becomes the primary behavioural objective.

Navigation should preserve orientation whenever practical.

Consequences

Future interaction systems should favour:

- evolution
 - continuity
 - progressive change
- over abrupt replacement.

ADR-060

Title

Separate Focus Transitions From Context Transitions

Status

Accepted

Context

Early exploration treated Focus and Context as identical interaction events.

Practical examples demonstrated that users frequently change Context without changing Focus.

Decision

Focus and Context transitions become independent behavioural concepts.

Consequences

Composition may evolve significantly while preserving the same Focus.

This produces calmer interactions.

ADR-061

Title

Treat Movement As Behavioural Communication

Status

Accepted

Context

Many modern interfaces use animation as decoration.

Founder workshops consistently described movement as a mechanism for preserving understanding.

Decision

Movement exists only to communicate behavioural change.

Animation becomes an implementation concern.

Consequences

Future motion systems should explain behaviour rather than entertain users.

ADR-062

Title

Composition Evolves Rather Than Rebuilds

Status

Accepted

Context

Traditional interfaces frequently discard one layout before constructing another.

This weakens continuity.

Decision

Compositions should evolve progressively.

Information should gain or lose emphasis naturally.

Consequences

Users should rarely feel they have entered an entirely new interface.

ADR-063

Title

Interaction Begins With User Intent

Status

Accepted

Context

Many interaction models begin with interface events such as clicks, taps or gestures.

These are implementation details.

Decision

Interaction begins when user intent changes.

Interface events merely communicate that intent to the platform.

Consequences

Future interaction systems remain implementation independent.

The same behavioural model can support:

- desktop
- television

- mobile
- voice
- future interaction paradigms

without modification.

ADR-064

Title

Treat Time As A Behavioural Input

Status

Accepted

Context

Entertainment changes continuously even when the user is absent.

Traditional software often ignores this until manually refreshed.

Decision

Time becomes a first-class behavioural input.

World evolution may therefore occur independently from direct user interaction.

Consequences

Future systems should support:

- scheduled evolution
- release events
- background preparation
- passive updates

while preserving the user's understanding.

ADR-065

Title

Separate User Behaviour From System Behaviour

Status

Accepted

Context

Internal engineering behaviour frequently leaks into user experience.

Examples include:

- provider refresh
- cache rebuilding
- synchronisation
- indexing

Decision

System behaviour should remain internal.

Only user behaviour should become part of the Interaction Model.

Consequences

Users continue experiencing:

- continuity
- trust
- predictability

without understanding implementation.

ADR-066

Title

Plugins Participate In Behaviour Rather Than Define Behaviour

Status

Accepted

Context

Allowing plugins to define independent interaction models fragments product identity.

Decision

Plugins contribute:

- Information
- Relationships
- Capability

The platform remains responsible for:

- behaviour
- continuity
- movement
- interaction states

Consequences

Every extension naturally feels like part of the same product.

ADR Relationships

```
[mermaid]
flowchart TD
```

```
ADR058["World Evolution"]
```

```
ADR059["Continuity"]
```

```
ADR060["Focus / Context"]
```

```
ADR061["Movement"]
```

```
ADR062["Composition"]
```

```
ADR063["User Intent"]
```

```
ADR064["Time"]
```

```
ADR065["User Behaviour"]
```

```
ADR066["Platform Behaviour"]
```

```
ADR058 --> ADR059
```

```
ADR059 --> ADR060
```

```
ADR060 --> ADR062
```

```
ADR062 --> ADR061
```

```
ADR063 --> ADR060
```

```
ADR064 --> ADR062
```

```
ADR065 --> ADR061
```

```
ADR066 --> ADR062
```

These decisions intentionally reinforce one another.

No behavioural decision should be interpreted independently from the others.

Future ADRs

Future Interaction Model ADRs are expected to cover:

- Shared experiences
- Multi-user interaction
- Collaborative viewing
- Cross-device continuity
- External player integration
- Adaptive interaction heuristics
- Ambient interaction

These concepts intentionally remain outside the scope of MDL-004 Version 0.1.

ADR Governance

Behavioural ADRs should change infrequently.

Changes should occur only when:

- user research demonstrates behavioural confusion,
- the Mental Model evolves,
- multiple specifications repeatedly conflict,
- significant architectural evidence supports an alternative.

Implementation convenience is not sufficient justification for modifying interaction behaviour.

Behaviour should remain significantly more stable than implementation.

Summary

The ADRs contained within MDL-004 define the behavioural identity of Mosaic.

Future implementations should treat these decisions as long-term behavioural constraints rather than implementation suggestions.

Together they establish how the user's World evolves while preserving continuity, trust and understanding.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The purpose of this chapter is to help contributors apply the Mosaic Interaction Model consistently.

Unlike the Mental Model, which explains **what exists**, the Interaction Model explains **how those concepts behave**.

Every contributor implementing behaviour should begin here before writing interface code.

Whether working on:

- backend services
- composition engines
- GraphQL
- Flutter
- HTMX
- web
- mobile
- television

the behavioural expectations remain identical.

Design Behaviour First

One of the most common mistakes in software development is implementing interaction before understanding behaviour.

Poor process.

Button

↓

Click

↓

Animation

↓

Done

Preferred process.

User Intent

↓

Behaviour

↓

Composition



Expression



Presentation



Implementation

Behaviour should always precede interface.

Think In Behaviour

Avoid describing features using interface terminology.

Poor.

Open Details Page

Better.

Shift Focus

Poor.

Show Dialog

Better.

Request Confirmation

Poor.

Navigate Home

Better.

Return To Previous Context

Behaviour should describe the user's experience.

Not implementation.

Preserve Continuity

Every contribution should preserve continuity.

Ask:

If the user disappears for thirty seconds...

...will they still understand what happened when they return?

Good interactions preserve:

- Focus
- Context

- Progress
- Relationships
- Understanding

Poor interactions reset them.

Design Around Intent

Every interaction begins with user intent.

Before implementation ask:

What is the user trying to achieve?

Not:

What interface should appear?

This distinction should influence:

- API design
- state management
- composition
- motion
- navigation

Intent always precedes implementation.

Avoid Page Thinking

One of the easiest mistakes contributors can make is reverting to traditional page-based thinking.

Avoid asking:

- Which page?
- Which route?
- Which screen?

Instead ask:

- Which World?
- Which Focus?
- Which Context?
- Which Composition?

The interface should emerge naturally from those answers.

Behaviour Is Shared

Interaction behaviour should remain consistent across every Mosaic client.

Desktop.

Television.

Mobile.

Tablet.

Future devices.

Presentation may differ.

Behaviour should not.

If contributors find themselves inventing different behavioural models for different clients, the implementation should be reconsidered.

Think In Evolution

Whenever information changes, ask:

How should the World evolve?

Not:

Which interface should replace the current one?

Good behaviour evolves.

Poor behaviour replaces.

This distinction should become instinctive.

Plugins

Plugin authors should never implement behavioural models.

Instead they contribute:

- Information
- Relationships
- Capability

The platform determines:

- Focus transitions
- Context transitions
- Composition evolution
- Movement
- Presentation

This separation allows every extension to feel native.

Preserve Behavioural Identity

Before introducing any new interaction ask:

Does this interaction reinforce:

- continuity
- understanding
- companionship
- calmness

or merely introduce novelty?

Novel interaction should never become a goal in itself.

Understanding remains the objective.

Common Mistakes

The following usually indicate contributors are bypassing the Interaction Model.

Treating Navigation As Behaviour

Users care about changing intent.

Not routes.

Behaviour Driven By Components

Components should communicate behaviour.

They should never define it.

Behaviour Driven By Frameworks

Framework limitations should never redefine interaction.

Technology adapts to behaviour.

Behaviour does not adapt to technology.

Plugin Behaviour

Plugins introducing:

- navigation
- custom interaction

- independent animation
- competing behavioural models

This fragments the product.

Review Questions

Before implementation ask:

- Does this preserve continuity?
- Does this strengthen understanding?
- Does this reduce cognitive effort?
- Would users naturally predict this behaviour?
- Is the World evolving...

or being replaced?

- Would this behaviour remain valid regardless of implementation technology?

If the answers remain unclear...

Return to the Interaction Model.

Behaviour Checklist

Every substantial behavioural proposal should satisfy the following.

- ☐ User intent is understood.
- ☐ Focus behaves consistently.
- ☐ Context evolves naturally.
- ☐ Composition changes progressively.
- ☐ Continuity is preserved.
- ☐ Behaviour remains platform independent.
- ☐ Plugins participate without redefining interaction.
- ☐ Presentation communicates rather than invents behaviour.

Final Guidance

Users should never think:

"That interface changed."

They should instead think:

"That makes sense."

If contributors consistently optimise for understanding rather than interface, Mosaic will gradually become easier to use without users ever consciously noticing why.

That is the ultimate objective of the Interaction Model.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the behavioural terminology introduced by **MDL-004 — Interaction Model**.

Unlike the glossary contained within **MDL-003**, which defines conceptual objects, this glossary defines behavioural concepts.

These terms should be used consistently throughout every future MDL and MDS specification.

Behavioural terminology is considered part of the Mosaic Design Language.

Changing terminology changes how contributors think about interaction.

B

Behaviour

The observable evolution of the user's World over time.

Behaviour exists independently from implementation.

Behaviour answers:

What should happen?

Implementation answers:

How does it happen?

Behavioural Continuity

The preservation of understanding as the user's World changes.

Continuity is considered one of the primary objectives of the Interaction Model.

Behavioural Weight

A conceptual measure of how significant an interaction is.

Weight is determined by conceptual change rather than visual change.

Example.

Episode

↓

Next Episode

Low weight.

Anime

↓

High weight.

Behavioural weight influences future composition and motion systems.

C

Composition Evolution

The continuous refinement of a Composition as the user's World changes.

Unlike page replacement, Composition Evolution preserves continuity.

Context Transition

A behavioural transition in which Context changes while Focus may remain the same.

Context Transitions are expected to occur significantly more frequently than Focus Transitions.

Continuity

The preservation of user understanding throughout interaction.

Users should feel they remain inside one evolving World rather than repeatedly entering new interfaces.

F

Focus Transition

A behavioural transition where the user's primary attention changes from one subject to another.

Focus Transitions should preserve understanding rather than reset it.

I

Interaction

The behavioural evolution of the user's World in response to:

- user intent
- time
- information
- relationships

Interaction is not synonymous with animation or interface.

Interaction Model

The behavioural architecture governing how concepts defined by the Mental Model evolve over time.

The Interaction Model intentionally remains independent from presentation.

Interaction State

The current behavioural mode of the user's World.

Examples include:

- Watching
- Reading
- Exploring
- Continuing
- Managing

Interaction States describe behaviour rather than interface.

L

Local Evolution

Behavioural changes affecting only the relevant portion of the current Composition.

Local evolution preserves continuity by avoiding unnecessary global reorganisation.

M

Movement

The communication of behavioural change.

Movement exists to explain:

- continuity
- hierarchy
- progression
- understanding

Movement is not decoration.

P

Progressive Evolution

A behavioural strategy in which significant changes occur through multiple understandable transitions rather than abrupt replacement.

T

Temporal Behaviour

Behaviour driven by the passage of time rather than direct user interaction.

Examples include:

- episode releases
- scheduled events
- countdown completion
- background updates

Temporal Behaviour allows the World to evolve naturally while users are absent.

Transitional State

A short-lived Interaction State existing only to support another state.

Examples include:

- Preparing Playback
- Searching
- Authenticating

Transitional States should never become destinations.

U

User Behaviour

Behaviour originating from user intention.

Examples include:

- Continue Watching
- Resume Reading
- Explore Cast

User Behaviour defines goals.

It does not define implementation.

S

System Behaviour

Behaviour performed internally by the platform in order to support User Behaviour.

Examples include:

- Composition recalculation
- Relationship updates
- Information processing

System Behaviour should remain largely invisible.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion, Immersion
MDL-002 Principles	Behavioural Decision Making
MDL-003 Mental Model	World, Focus, Context
MDL-005 Composition Model	Composition Behaviour
MDS Motion System	Behaviour Communication

Terminology Rules

Future contributors should:

- describe behaviour before interface
- prefer conceptual terminology over implementation terminology
- avoid framework-specific language
- preserve behavioural consistency across specifications

Interaction terminology should remain stable throughout the lifetime of the Mosaic Design Language.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the references and architectural influences that informed **MDL-004 — Interaction Model**.

The purpose of these references is to provide architectural context for contributors rather than implementation requirements.

MDL is intentionally opinionated.

External references inform the design language.

They do not define it.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Product Behaviour 3. Human Factors 4. Motion & Interaction 5. Systems Architecture

The Mosaic Design Language always remains the authoritative source.

Internal References

MDL-001 — Vision

Provides:

- Product vision
- Companion philosophy
- Immersion
- Long-term goals

Every behavioural decision defined by MDL-004 should reinforce the vision established by MDL-001.

MDL-002 — Principles

Provides:

- Behavioural decision framework
- Design governance
- Principle hierarchy

MDL-004 should be interpreted as the behavioural implementation of MDL-002.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Information
- Relationships
- Composition
- Expressions

MDL-004 intentionally assumes these concepts already exist.

It defines how they evolve over time.

Future Specifications

The following specifications depend directly upon the Interaction Model.

- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-005 Motion System
- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications should extend behaviour rather than redefine it.

Human Factors

Mental Models

The Interaction Model assumes users continually build expectations about system behaviour.

Interaction should reinforce those expectations.

Unexpected behaviour should remain rare.

Cognitive Load

Reducing cognitive effort remains one of the primary behavioural objectives.

Behaviour should minimise:

- orientation effort
- context switching
- uncertainty

- unnecessary decision making

Spatial Memory

The Interaction Model intentionally preserves:

- continuity
- object permanence
- behavioural consistency

Users should rarely need to reconstruct understanding after every interaction.

Interaction Design

The following themes significantly influenced MDL-004.

Continuity

Interfaces should evolve rather than restart.

Progressive Disclosure

Information should become available as it becomes useful.

Direct Manipulation

Users should feel that they are interacting with their World rather than operating software.

Behaviour Before Animation

Motion communicates behaviour.

It should never invent behaviour.

Systems Thinking

MDL-004 intentionally models interaction as a system rather than isolated events.

Key ideas include:

- continuous evolution
- state transitions
- behavioural hierarchy
- conceptual consistency

Future implementations should preserve these ideas regardless of technology.

Behavioural Architecture

The Interaction Model intentionally separates:

Intent

↓

Behaviour

↓

Presentation

rather than:

Input

↓

Animation

↓

Output

This distinction allows future client implementations to evolve independently while preserving a consistent behavioural language.

Design Systems

The organisation of MDL draws inspiration from mature design systems that separate:

- philosophy
- principles
- conceptual models
- behaviour
- implementation

rather than treating interaction as a collection of component states.

Behaviour should remain platform-independent wherever practical.

Software Architecture

Although MDL intentionally avoids implementation, the following architectural ideas influenced its structure.

- Separation of Concerns
- Event-Driven Systems
- State Machines
- Architecture Decision Records
- Documentation as Code

These influenced documentation organisation rather than behavioural philosophy.

Mosaic-Specific Influences

The Interaction Model emerged directly from founder discovery workshops.

Major behavioural discoveries included:

- Worlds evolve rather than reset.
- Focus changes are more important than navigation.
- Context changes are more common than Focus changes.
- Movement exists to preserve understanding.
- Composition evolves continuously.
- Plugins participate in behaviour rather than defining it.

These discoveries distinguish the Mosaic Interaction Model from traditional page-based applications.

Normative References

Required reading before implementing MDL-004.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model

Informative References

Future contributors may also wish to review:

- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-005 Motion System
- MDS-007 Tile Framework

These documents describe how the behavioural concepts established by MDL-004 become implementation.

Living Document

This reference list should remain intentionally concise.

References should only be added when they materially influence:

- behavioural architecture
- interaction philosophy

- governance
- terminology

The purpose of this document is to preserve architectural reasoning rather than provide an exhaustive bibliography.

Completion

This concludes **MDL-004 — Interaction Model**.

The next specification in the Mosaic Design Language is:

MDL-005 — Composition Model

Where MDL-004 defines **how the user's World behaves**, MDL-005 defines **how that World is organised into meaningful, adaptive compositions**.

MDL-005 formalises concepts including:

- hierarchy
- adaptive composition
- anchors
- priorities
- density
- breathing space
- composition solving
- information organisation

It establishes the conceptual bridge between interaction and the Mosaic Design System.